

Alona OS

Reference Design για IoT, resources, tasks, money, maintenance και automations

Σκοπός

Να υπάρχει ένα ενιαίο reference για το πώς δένουν οι μετρήσεις από ESP32 και smart devices με domains όπως Water, Energy, Food Production, Finance, Tasks, Maintenance, Protocols, Automations και Security.

Βασική αρχή

Το app δεν είναι απλό IoT dashboard. Είναι ένα προσωπικό operating system για σπίτι, κτήμα, παραγωγή, resources και εργασίες. Όλα συνδέονται μέσα από κοινά entities, events και links.

Περιεχόμενα

- 1. Product model και mental model
- 2. Core database schema
- 3. Measurement και IoT layer
- 4. Resources, expenses και planning
- 5. Tasks, maintenance, protocols και automations
- 6. UX structure και first features
- 7. Prompt για v0
- 8. Πρόταση βάσης δεδομένων

1. Product model και mental model

Το σύστημα πρέπει να δουλεύει με functional navigation, επειδή αυτό είναι πιο φυσικό για καθημερινή χρήση. Παράλληλα πρέπει κάθε λειτουργία να μπορεί να φιλτράρει τα δεδομένα της ανά χώρο, asset ή project.

Main idea:

- Navigate by function: Energy, Water, Food Production, Tasks, Finance, Maintenance
- Filter by context: House, Garden, Tank, Battery, Room, Vehicle, Hive
- Connect everything through entities, links and events

Τα βασικά concepts

Concept	Ρόλος
Domain	Λειτουργική περιοχή όπως Energy, Water, Food Production, Security ή Finance.
Location	Φυσικός χώρος όπως House, Bedroom, Energy Room, Well Area ή Garden Area.
Entity	Οτιδήποτε αξίζει να συνδεθεί με μετρήσεις, tasks, expenses, events ή maintenance.
Event	Κάτι που συνέβη και πρέπει να μπει στο timeline.
Resource	Κάτι που παράγεται, αποθηκεύεται, καταναλώνεται, αγοράζεται ή μεταφέρεται.
Task	Ενέργεια που πρέπει να γίνει από εσένα ή να προκύψει από rule, protocol ή maintenance.
Automation	Workflow που παίρνει αποφάσεις, κάνει checks, δημιουργεί tasks ή στέλνει commands.
Protocol	Mode του συστήματος και checklist ενεργειών για συγκεκριμένη κατάσταση.

Προτεινόμενα top-level domains

Command Center
Resources
Energy
Water
Environment
Food Production
Security
Tasks
Maintenance
Automations
Protocols
Finance
Timeline
Settings

Food Production ως παράδειγμα επέκτασης

Τα μελίσσια δεν χρειάζεται να υπάρχουν από την αρχή. Το σωστό είναι να προβλεφθεί ότι κάτω από Food Production μπορούν να μπουν Garden, Herbs, Fruit Trees, Beehives, Food Storage και αργότερα Food Consumption.

Food Production

```
Garden
  Raised Bed 1
  Tomatoes 2026
  Watering System
Herbs
Fruit Trees
Future: Beehives
  Hive 1
  Hive 2
Future: Food Storage
Future: Food Consumption
```

2. Core database schema

Ο πυρήνας της βάσης πρέπει να είναι generic. Αυτό αποφεύγει το πρόβλημα να έχεις ξεχωριστά και ασύνδετα modules για Garden, Water, Finance και Tasks.

domains

```
domains
- id
- name
- slug
- description
- parent_domain_id
- status
```

locations

```
locations
- id
- name
- type
- parent_location_id
- description
```

entities

```
entities
- id
- name
- entity_type
- primary_domain_id
- location_id
- parent_entity_id
- status
- installed_at
- retired_at
- notes
- metadata
```

Παραδείγματα entity_type: area, asset, device, sensor, resource_store, resource_system, crop, vehicle, tool, integration, project.

entity_links

```
entity_links
- id
- source_entity_id
- target_entity_id
- relation_type
- valid_from
- valid_to
- notes
```

Τα entity links είναι το κλειδί για να μπορεί το Garden να χρησιμοποιεί Water, να έχει Expenses, να σχετίζεται με Watering System και να επηρεάζεται από Automations χωρίς να φτιάχνεις hardcoded σχέσεις.

Παράδειγμα σχέσης	Σημασία
Garden uses Water Tank	To Garden μπορεί να δείξει νερό που σχετίζεται με αυτό.
Pump fills Water Tank	To Water domain ξέρει ποιο asset γεμίζει τη δεξαμενή.
Generator charges Battery Bank	To Energy domain βλέπει εξάρτηση generator και battery.
Soil Sensor monitors Raised Bed 1	To sensor συνδέεται με το πραγματικό αντικείμενο που μετράει.
Automation controls Pump	Η automation έχει target asset και μπορεί να γράψει timeline.

3. Measurement και IoT layer

Τα inputs μπορεί να είναι ESP32 nodes, Victron values, weather data, smart devices ή manual entries. Όλα πρέπει να μπαίνουν στο ίδιο abstraction: data source, device, sensor, metric, measurement stream και measurements.

data_sources

```
data_sources
- id
- name
- source_type
- integration_type
- status
- last_seen_at
- metadata
```

devices

```
devices
- id
- entity_id
- data_source_id
- device_type
- manufacturer
- model
- firmware_version
- status
```

- last_seen_at

sensors

- ```
sensors
```
- id
  - entity\_id
  - device\_id
  - sensor\_type
  - status
  - calibration\_data
  - installed\_at

## metric\_definitions

- ```
metric_definitions
```
- id
 - name
 - unit
 - value_type
 - category

measurement_streams

- ```
measurement_streams
```
- id
  - name
  - metric\_id
  - source\_entity\_id
  - subject\_entity\_id
  - data\_source\_id
  - unit
  - sampling\_interval\_seconds
  - aggregation\_type
  - is\_active

## measurements and current\_values

- ```
measurements
```
- id
 - stream_id
 - measured_at
 - value_number
 - value_text
 - value_boolean
 - quality
 - raw_payload
-
- ```
current_values
```
- stream\_id
  - latest\_value
  - latest\_value\_text
  - measured\_at
  - quality

### Γιατί χρειάζεται measurement\_stream;

Επειδή το ίδιο sensor type μπορεί να μετράει διαφορετικό subject. Άλλο είναι ο αισθητήρας σαν συσκευή και άλλο το πράγμα που παρακολουθεί, για παράδειγμα Bedroom ή Water Tank.

### States και commands

```
entity_states
- entity_id
- state_key
- state_value
- updated_at
- source_event_id

state_history
- id
- entity_id
- state_key
- old_value
- new_value
- changed_at
- source_type
- source_id

command_definitions
- id
- target_entity_id
- command_name
- parameters_schema
- requires_confirmation
- safety_level

command_requests
- id
- command_definition_id
- target_entity_id
- requested_by
- requested_at
- status
- parameters
- reason
- source_automation_run_id

command_results
- id
- command_request_id
- status
- executed_at
- response_payload
- verification_status
- verified_at
- error_message
```

## 4. Resources, expenses και planning

Resources είναι τα πράγματα που σε νοιάζει να παρακολουθείς ως παραγωγή, κατανάλωση, απόθεμα ή ιστορικό. Για το δικό σου use case αυτό περιλαμβάνει electricity, water, petrol, gas και αργότερα food ή honey.

## resource\_types, resource\_stores και resource\_flows

```
resource_types
- id
- name
- base_unit
- category

resource_stores
- id
- entity_id
- resource_type_id
- capacity
- unit
- current_estimated_amount
- min_safe_amount
- max_safe_amount

resource_flows
- id
- resource_type_id
- flow_type
- quantity
- unit
- from_entity_id
- to_entity_id
- related_entity_id
- source_type
- measured_by_stream_id
- occurred_at
- confidence
- notes
```

| Resource flow | Παράδειγμα                                              |
|---------------|---------------------------------------------------------|
| produced      | PV produced 12 kWh.                                     |
| consumed      | Garden consumed 300 L water.                            |
| purchased     | Bought 20 L petrol.                                     |
| transferred   | Well filled Water Tank with 500 L.                      |
| estimated     | Estimated garden water usage when no flow meter exists. |
| corrected     | Manual correction after checking physical stock.        |

## expenses και allocations

```
expenses
- id
- date
- title
- description
- amount
- currency
- vendor
- payment_method
- category_id
- invoice_file_id
- notes
```

```
expense_categories
- id
- name
- parent_category_id

expense_allocations
- id
- expense_id
- entity_id
- domain_id
- resource_type_id
- amount
- percentage
- notes
```

Το expense μπορεί να είναι απλό λογιστικό log ή να συνδέεται με domain/entity/resource. Για παράδειγμα η ασφάλεια αυτοκινήτου μπορεί να συνδεθεί με entity Car. Τα seeds μπορούν να συνδεθούν με Garden. Το petrol μπορεί να μοιραστεί σε Generator και Tools.

## budgets και forecasts

```
budgets
- id
- name
- period_start
- period_end
- amount
- currency
- domain_id
- entity_id
- resource_type_id
- category_id

forecasts
- id
- forecast_type
- domain_id
- entity_id
- resource_type_id
- period_start
- period_end
- predicted_value
- unit
- confidence
- source_type
- created_at
```

## 5. Tasks, maintenance, protocols και automations

Τα tasks είναι actionable reminders. Δεν είναι τα ίδια workflows. Μπορούν όμως να δημιουργούνται από workflows, protocols, maintenance plans, observations ή thresholds.

### tasks

```
tasks
- id
- title
- description
- status
```



- priority
- due\_at
- scheduled\_for
- completed\_at
- source\_type
- source\_id
- recurrence\_rule
- estimated\_duration\_minutes
- calendar\_event\_id

#### task\_links

- id
- task\_id
- entity\_id
- relation\_type

#### task\_checklist\_items

- id
- task\_id
- title
- status
- sort\_order

## maintenance

#### maintenance\_plans

- id
- entity\_id
- title
- description
- interval\_type
- interval\_value
- next\_due\_at
- next\_due\_metric\_value
- active

#### maintenance\_records

- id
- entity\_id
- maintenance\_plan\_id
- title
- description
- performed\_at
- performed\_by
- result
- next\_due\_at
- cost\_total
- notes

## protocols

#### protocols

- id
- name
- description
- protocol\_type
- default\_duration
- is\_system\_mode
- status

#### protocol\_activations

```
- id
- protocol_id
- status
- activated_at
- deactivated_at
- activated_by
- reason
- notes

protocol_task_templates
- id
- protocol_id
- title
- description
- priority
- offset_from_activation
- target_entity_id

protocol_rules
- id
- protocol_id
- rule_type
- target_entity_id
- config
```

## automations as lifecycles

```
automations
- id
- name
- description
- status
- automation_type
- safety_level
- requires_confirmation
- primary_domain_id

automation_triggers
- id
- automation_id
- trigger_type
- config

automation_conditions
- id
- automation_id
- condition_type
- config
- sort_order

automation_steps
- id
- automation_id
- step_type
- config
- sort_order
- on_failure

automation_runs
- id
- automation_id
- status
```

```
- started_at
- completed_at
- trigger_event_id
- result
- summary

automation_run_steps
- id
- automation_run_id
- automation_step_id
- status
- started_at
- completed_at
- input_payload
- output_payload
- error_message
```

## events και unified timeline

```
events
- id
- event_type
- severity
- title
- description
- occurred_at
- source_type
- source_id
- actor_type
- actor_id
- payload

event_links
- id
- event_id
- entity_id
- relation_type

observations
- id
- title
- body
- observed_at
- created_at
- severity
- observation_type
- created_by

observation_links
- observation_id
- entity_id
- relation_type
```

## 6. UX structure και first features

Το UX πρέπει να δουλεύει και σαν dashboard και σαν command center. Η αρχική οθόνη δεν πρέπει απλά να δείχνει charts. Πρέπει να απαντάει στο ερώτημα: τι συμβαίνει, τι θέλει προσοχή και τι πρέπει να κάνω;

## Command Center

### System Status

- active protocol or mode
- active alerts
- critical resources
- today tasks
- automation activity
- security status

### Resources Snapshot

- battery SOC
- PV production
- current load
- water tank level
- indoor temperatures

### Needs Attention

- overdue tasks
- low resources
- failed automations
- maintenance due
- abnormal readings

### Recent Timeline

- latest events
- latest commands
- latest expenses
- latest observations

## Entity detail pages

Κάθε entity πρέπει να έχει contextual tabs. Έτσι ανοίγεις Garden και βλέπεις μαζί water, money, tasks, maintenance, automations και timeline που αφορούν το Garden.

### Example: Garden page

- Overview
- Live / Measurements
- Water
- Tasks
- Expenses
- Maintenance
- Automations
- Protocols
- Observations
- Timeline
- Settings

### Example: Water Tank page

- Overview
- Current Level
- History
- Related Devices
- Resource Flows
- Tasks
- Maintenance
- Alerts
- Timeline

## First implementation scope

Η πρώτη έκδοση πρέπει να είναι μικρή αλλά να χρησιμοποιεί το σωστό foundation.

| Feature     | Πρώτη έκδοση                                                                 |
|-------------|------------------------------------------------------------------------------|
| Energy      | Victron-like snapshot με battery SOC, PV power, load power και battery flow. |
| Water       | Water tank level, well/pump placeholder και history chart.                   |
| Environment | 3 indoor rooms με temperature και humidity cards.                            |
| Timeline    | Unified recent events από measurements, alerts, tasks και manual logs.       |
| Tasks       | Manual tasks, due dates, status και links σε entities.                       |
| Expenses    | Simple expense log με allocations σε domain/entity.                          |
| Entities    | House, rooms, water tank, battery, PV, garden placeholder.                   |
| Settings    | Sensors, streams, thresholds και notification preferences as placeholders.   |

## MVP tables

```
domains
locations
entities
entity_links

data_sources
devices
sensors
metric_definitions
measurement_streams
measurements
current_values

events
event_links
observations
observation_links

tasks
task_links
expenses
expense_allocations

resource_types
resource_stores
resource_flows

entity_states
state_history
```

## 7. Prompt για v0

Το παρακάτω prompt είναι για να σχεδιάσει το v0 την πρώτη έκδοση του UI. Ζήτα του React dashboard με mock data και όχι backend implementation.

Create a modern web app UI called "Alona OS". It is a personal operating system for an off-grid house and small farm. It combines IoT monitoring, resources, tasks, maintenance, expenses, automations, protocols and security.

Design only the first MVP features. Use realistic mock data. Do not build backend logic. Focus on layout, UX and component structure.

Product concept:

- The app is not just an IoT dashboard.
- It is a command center for the house.
- Navigation is mainly by function: Command Center, Resources, Energy, Water, Environment, Food Production, Tasks, Maintenance, Automations, Protocols, Finance, Timeline and Settings.
- Every object is an entity that can have measurements, events, tasks, expenses, maintenance, observations and automations.
- The first MVP monitors electricity, water and indoor temperatures in 3 rooms.

Visual style:

- Clean, calm, practical and slightly technical.
- Use a neutral light theme.
- Use cards, tables, charts and status badges.
- Make it suitable for a mountain/off-grid house.
- Use responsive layout for desktop and tablet.

Main layout:

- Left sidebar navigation.
- Top bar with current mode, date/time, active alerts and quick action button.
- Main content area.
- A right-side panel or bottom section for "Needs Attention" and recent timeline.

Pages to design:

### 1. Command Center

Show:

- System mode: Normal
- Battery SOC card
- PV production card
- Current load card
- Water tank level card
- Indoor environment summary for Living Room, Bedroom and Bathroom
- Active alerts
- Today tasks
- Needs attention
- Recent timeline
- Quick actions: Add task, Log expense, Add observation, Activate protocol

### 2. Energy page

Show:

- Battery SOC
- PV power
- House load
- Battery charge/discharge
- Generator placeholder
- Chart for SOC history
- Table of recent energy events
- Automation card: "Low battery workflow"

### 3. Water page

Show:

- Water tank level
- Estimated liters available
- Well status placeholder
- Pump status placeholder
- Daily water usage chart
- Alerts and tasks related to water

- Resource flow cards: received, consumed, estimated usage

#### 4. Environment page

Show:

- Three room cards: Living Room, Bedroom, Bathroom
- Temperature and humidity values
- Small trend indicators
- Sensor health and last seen time
- Chart comparing room temperatures

#### 5. Food Production page

This is mostly a future-ready placeholder.

Show:

- Garden entity card
- Future sections: Herbs, Fruit Trees, Beehives, Food Storage, Food Consumption
- Garden tabs: Overview, Water, Tasks, Expenses, Observations, Timeline
- Mock garden task and mock expense

#### 6. Tasks page

Show:

- Task list grouped by Today, Upcoming and Overdue
- Filters by domain and entity
- Task detail drawer with linked entities, source and checklist
- Tasks can be manual or generated from protocol, automation or maintenance

#### 7. Finance page

Show:

- Expense log
- Expense categories
- Allocations to domain/entity/resource
- Example expenses: car insurance, garden seeds, water filter, petrol for generator
- Summary cards: monthly total, garden cost, water cost, energy cost

#### 8. Timeline page

Show a unified timeline with different event types:

- measurement received
- threshold crossed
- task completed
- expense logged
- protocol activated
- automation run
- command result
- security event
- observation added

#### 9. Settings page

Show placeholder sections:

- Entities
- Sensors
- Measurement streams
- Thresholds
- Notification channels
- Protocols
- Automations

Important data model terms to reflect in the UI:

- Domain
- Location
- Entity
- Resource
- Measurement Stream
- Event
- Task
- Expense Allocation

- Protocol
- Automation Run
- Observation

Use mock components and mock data. Make the UI feel like a real MVP that can later connect to a database and real IoT data.

## 8. Πρόταση βάσης δεδομένων

### Πρόταση

Χρησιμοποίησε PostgreSQL ως βασική βάση από την αρχή. Για τις μετρήσεις μπορείς αρχικά να μείνεις σε απλά indexed tables ή partitioned tables. Αν ο όγκος των time-series γίνει μεγάλος, πρόσθεσε TimescaleDB extension αργότερα.

### Γιατί PostgreSQL

- Το schema έχει πολλά relational links: entities, events, expenses, tasks, maintenance, automations και resources.
- Θα χρειαστείς δυνατά queries για cross-domain views όπως Garden cost, Water usage, Energy timeline ή tasks by entity.
- Το JSONB βοηθάει για metadata, raw\_payload, config, rule definitions και automation steps χωρίς να χαλάς το relational core.
- Μπορεί να σηκώσει historical measurements καλύτερα από SQLite όταν αυξηθούν οι streams και τα dashboards.
- Υποστηρίζει indexes, materialized views, constraints και migrations με ασφάλεια.

### Πρακτικό setup

Recommended setup:

- PostgreSQL as the source of truth
- JSONB for flexible configs and raw payloads
- Indexed measurements table for MVP
- Optional partitioning by time for measurements
- Optional TimescaleDB when measurement volume grows
- Keep current\_values as fast dashboard cache
- Keep events as the unified audit/timeline layer

### Πότε θα διάλεγα SQLite

SQLite είναι καλή επιλογή αν θες εντελώς local, μικρό, απλό και single-user σύστημα με λίγες μετρήσεις. Επειδή όμως εδώ ο στόχος είναι operating system με πολλά cross-domain links, automations, resource history, finance allocations και time-series growth, PostgreSQL είναι η πιο σωστή βάση.

### TimescaleDB ή όχι;

Δεν θα ξεκινούσα αναγκαστικά με TimescaleDB ως πρώτη εξάρτηση. Θα σχεδιάζα το measurements table έτσι ώστε να μπορεί να μετατραπεί σε hypertable ή partitioned table χωρίς αλλαγή στο domain model. Αν οι μετρήσεις μείνουν λίγες, Postgres μόνο αρκεί. Αν αρχίσεις να κρατάς πολλά streams ανά λεπτό για χρόνια, τότε TimescaleDB αξίζει.

### Τελική επιλογή

Use PostgreSQL first.  
Use JSONB selectively for flexible metadata and automation configs.  
Use relational tables for the core model.



Use `current_values` for dashboards.  
Add TimescaleDB later if measurements become the bottleneck.

## Final reference summary

Το σημαντικότερο design decision είναι να μη χτίσεις απομονωμένα modules. Χτίσε κοινό entity graph, unified events, generic tasks, generic expenses, resource flows και measurement streams. Μετά τα domains είναι views πάνω στο ίδιο σύστημα.

Everything important is an Entity.  
Everything that happens is an Event.  
Everything measurable is a Measurement Stream.  
Everything actionable is a Task or Command.  
Everything planned is a Protocol, Schedule, Budget or Forecast.  
Everything consumed or produced is a Resource Flow.

## 9. Repository structure και boundaries

Η εφαρμογή μπορεί να χωριστεί σε δύο βασικά repos: ένα Elixir umbrella repo για το core σύστημα και ένα firmware repo για τον κώδικα των συσκευών. Ο στόχος είναι καθαρά boundaries: το domain model να μη μπλέκεται με το web UI και το firmware να μη γνωρίζει λεπτομέρειες της βάσης.

### 9.1 Repos

| Repo              | Ρόλος                                                          | Περιεχόμενο                                                                           |
|-------------------|----------------------------------------------------------------|---------------------------------------------------------------------------------------|
| alona-os-core     | Elixir umbrella application για backend, ingestion και web UI. | apps/alona_core, apps/alona_ingest, apps/alona_web                                    |
| alona-os-firmware | Κώδικας για ESP32/devices και shared firmware utilities.       | device firmware, MQTT payload contracts, sensor readers, connectivity, power handling |

### 9.2 alona-os-core ως Elixir umbrella

Το repo alona-os-core πρέπει να είναι umbrella project. Μέσα στο apps/ υπάρχουν τρία apps με ξεκάθαρες ευθύνες.

| App          | Κύρια ευθύνη                                                                                                                            | Δεν πρέπει να κάνει                                                                                               |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| alona_core   | Domain model, Ecto schemas, contexts, events, tasks, expenses, resources, measurements, business rules και persistence.                 | Δεν πρέπει να έχει Phoenix UI, LiveViews ή MQTT-specific code.                                                    |
| alona_ingest | Ingestion από MQTT, Victron, ESP32 και άλλα integrations. Normalization payloads, validation, heartbeats, command adapters και workers. | Δεν πρέπει να αποφασίζει UI flows ή να περιέχει LiveView logic.                                                   |
| alona_web    | Phoenix endpoint, router, LiveViews, HEEx components, dashboards, forms, navigation και UX.                                             | Δεν πρέπει να γράφει απευθείας στη βάση παρακάμπτοντας τα contexts του alona_core ή να μιλά απευθείας με devices. |

### 9.3 Προτεινόμενη δομή alona-os-core

```
alona-os-core/
 mix.exs
 config/
 config.exs
 dev.exs
 prod.exs
 runtime.exs
 test.exs
 apps/
 alona_core/
 mix.exs
 lib/alona_core/
 application.ex
 repo.ex
 domains/
 locations/
 entities/
 measurements/
 resources/
 events/
 tasks/
 finance/
 maintenance/
 protocols/
 automations/
 commands/
 observations/
 priv/repo/
```

```

 migrations/
 seeds.exs
 test/

alona_ingest/
 mix.exs
 lib/alona_ingest/
 application.ex
 mqtt/
 client.ex
 subscriptions.ex
 topic_router.ex
 adapters/
 esp32_adapter.ex
 victron_adapter.ex
 smart_device_adapter.ex
 normalizers/
 measurement_normalizer.ex
 event_normalizer.ex
 heartbeat_normalizer.ex
 workers/
 measurement_writer.ex
 heartbeat_worker.ex
 commands/
 command_dispatcher.ex
 command_result_handler.ex
 health/
 test/

alona_web/
 mix.exs
 lib/alona_web/
 endpoint.ex
 router.ex
 telemetry.ex
 components/
 layout_components.ex
 metric_card.ex
 status_badge.ex
 timeline_item.ex
 entity_card.ex
 live/
 command_center_live.ex
 energy_live.ex
 water_live.ex
 environment_live.ex
 food_production_live.ex
 tasks_live.ex
 finance_live.ex
 timeline_live.ex
 settings_live.ex
 layouts/
 assets/
 priv/static/
 test/
test/

```

## 9.4 Data flow ανάμεσα στα apps

Η κατεύθυνση των δεδομένων πρέπει να είναι προβλέψιμη. Το ingestion γράφει normalized facts στο core. Το web διαβάζει από το core και κάνει actions μέσω core contexts.

```
Device / ESP32 / Victron / Smart device
-> alona_ingest
 -> parse topic / payload
 -> normalize measurement, event, heartbeat or command result
 -> call alona_core contexts
 -> write measurements, current_values, events, states
 -> publish updates through PubSub if needed
 -> alona_web LiveViews update dashboards
```

### Command flow για μελλοντική αποστολή εντολών:

```
alona_web
-> user requests action
-> alona_core creates command_request with status = pending/proposed
-> alona_ingest dispatches command through adapter
-> device responds or state changes
-> alona_ingest writes command_result
-> alona_core verifies state/measurement if needed
-> event is written to timeline
-> alona_web shows result
```

## 9.5 Boundaries και κανόνες σχεδιασμού

- Το alona\_core είναι source of truth για entities, measurements, current values, events, tasks, expenses και resource flows.
- Το alona\_ingest μεταφράζει εξωτερικό κόσμο σε εσωτερικό μοντέλο. Δεν πρέπει να έχει business UI assumptions.
- Το alona\_web είναι presentation layer. Δεν πρέπει να γνωρίζει MQTT topics, ESP32 payload quirks ή raw device protocols.
- Τα firmware payloads πρέπει να είναι versioned ώστε να μπορείς να αλλάξεις συσκευές χωρίς να σπάει το core.
- Τα commands πρέπει να περνούν από command\_request / command\_result lifecycle, όχι απευθείας από LiveView σε device.
- Αν αργότερα μπει LLM, πρέπει να διαβάζει από alona\_core και να προτείνει actions. Δεν πρέπει να εκτελεί απευθείας device commands χωρίς confirmation.

## 9.6 alona-os-firmware repo

Το firmware repo κρατά τον κώδικα για τα ESP32 nodes και οτιδήποτε τρέχει σε συσκευές. Δεν πρέπει να εξαρτάται από Phoenix ή από τη βάση. Η σύμβαση με το core γίνεται μέσω MQTT topics, payload schemas και device identity.

```
alona-os-firmware/
 README.md
 docs/
 hardware/
 room-node.md
 water-tank-node.md
 weather-node.md
 soil-node.md
 wiring/
 payloads/
 mqtt-topics.md
 payload-versioning.md
 shared/
 config/
 device_config.h
 connectivity/
 wifi_manager.h
 mqtt_client.h
```

```

protocol/
 mqtt_topics.h
 payloads.h
 device_identity.h
sensors/
 aht20_reader.h
 ds18b20_reader.h
 tank_level_reader.h
 soil_moisture_reader.h
power/
 battery_monitor.h
 deep_sleep.h
utils/
 json_helpers.h
 time_sync.h
devices/
 esp32-room-node/
 src/
 platformio.ini
 README.md
 esp32-water-tank-node/
 src/
 platformio.ini
 README.md
 esp32-weather-node/
 src/
 platformio.ini
 README.md
 esp32-soil-node/
 src/
 platformio.ini
 README.md
tests/

```

## 9.7 Firmware responsibilities

| Responsibility    | Περιγραφή                                                                                          |
|-------------------|----------------------------------------------------------------------------------------------------|
| Device identity   | Κάθε συσκευή έχει stable device_id, firmware_version, hardware_version και optional location hint. |
| Connectivity      | Wi-Fi/MQTT reconnects, backoff, retained status όπου χρειάζεται και Last Will/Testament.           |
| Sensor reading    | To firmware διαβάζει sensors και στέλνει raw/normalized values με quality flags.                   |
| Power behavior    | Battery voltage, sleep strategy, low-power modes και health payloads.                              |
| Payload contracts | Versioned payloads ώστε το alona_ingest να μπορεί να κάνει normalize χωρίς brittle parsing.        |
| Commands          | Αργότερα οι συσκευές μπορούν να δέχονται commands, αλλά πάντα με command id και result payload.    |

## 9.8 Πρώτη υλοποίηση με βάση το repo split

Για το MVP, ξεκίνα από το alona-os-core. Το firmware repo μπορεί αρχικά να έχει μόνο ένα room node και ένα water tank node ως reference implementations.

Phase 1

- alona\_core: domains, locations, entities, metrics, streams, current\_values, events, tasks, expenses, resources
- alona\_web: app shell, command center, energy, water, environment, timeline
- alona\_ingest: mock/manual ingestion first, MQTT adapter later

Phase 2

- alona\_ingest: MQTT subscriptions and ESP32 payload normalization
- alona-os-firmware: room node and water tank node

Phase 3

- Victron adapter
- command\_request / command\_result lifecycle
- protocols and automation runs